PO Governance — Sign-in by role, with every stored procedure call

Purpose: to find quickly *which stored procedure runs, when, and why* during sign-in and the first screens of each role. This helps when reading a SQL Profiler trace or checking a problem in production.

How to read the tables: each row is one step, in the order it happens.

- **API task:** the POGovApi action the portal calls (API_MASTER task name).
- **Procedure (@Cmd):** what POGovApi runs in SQL Server.
- A dash (–) means the step makes no database call.

---

1. The procedures in one table

Procedure	@Cmd used here	Table	What it is for
`PRO_POGOV_DOC_GET`	`GET_STAMP` (Name = users / sessions / manifest)	`doc_stamps`	"Has this list changed since I last read it?" A version number, 2 page reads. Runs very often.
	`GET_USERS`	`app_users`	Read all logins (only when the stamp changed)
	`GET_SESSIONS`	`app_sessions`	Read all sessions (only when the stamp changed)
	`GET_DOC` (Name = manifest / params / scr_mapping)	`app_settings`	Read one settings document
`PRO_POGOV_DOC_MAP_PUT`	`USERS`	`app_users`	Save changed logins (last sign-in, failed attempts, roles …)
	`SESSIONS`	`app_sessions`	Save new, ended or removed sessions
`PRO_POGOV_DOC_PUT`	–	`app_settings`	Save manifest / params / scr_mapping
`PRO_POGOV_AUDIT`	`APPEND` / `READ`	`audit_log`	Write / read audit lines (login, login-fail, logout, user-add …)
`PRO_POGOV_LOG_READ`	`READ` (Stream = requests, Day = today)	`request_events`	Today's PO requests (request number, today's list)
	`DAYS`	`request_events`	On which days were there requests
	`READ_DAYS`	`request_events`	Requests of several days (last 30 days for the approvals count)
`PRO_POGOV_LOG_APPEND`	–	`request_events`, `upload_log`, `upload_batches`, …	Add one line to a log
`PRO_POGOV_BLOB_GET`	`GET` (Kind = ds, Key = f1 …)	`dataset_versions`	Read one stored data file (gzip)

Procedure	@Cmd used here	Table	What it is for
`PRO_POGOV_APP_LOCK`	`GET` / `RELEASE`	(SQL Server app lock)	"Only one writer at a time" lock. Runs on its own connection named **`POGovernance (lock)`**.
`PRO_GET_ALL_API`	`ALL` / `BY_TASK` / `SERVER_TASK` / `BASE_URL`	`API_MASTER`, `API_BASE_URL`, `API_AUTHORIZATION`	API list for the portal, HRMS URL, portal base URL

Profiler tip:

- The lock calls appear under ApplicationName **`POGovernance (lock)`**. Filter `ApplicationName LIKE 'POGovernance%'` to see them.
- `Audit Login`, `Audit Logout` and `exec sp_reset_connection` are connection-pool bookkeeping (`EventSubClass = 2`, pooled), not user sign-ins.

---

2. Steps every role goes through

2.1 Opening the sign-in page

#	What happens	API task	Procedure	Why
1	Sign-in page asks how people sign in	`PO_HEALTH`	–	Shows "Employee ID" or "Email" (kept for 1 minute)
2	CAPTCHA picture is drawn	–	–	Kept in portal memory

2.2 Clicking Sign in (an employee: Employee ID + HRMS password)

#	What happens	API task	Procedure (@Cmd)	Why
1	Portal checks the CAPTCHA	–	–	Wrong → stop, nothing reaches the API
2	Portal sends ID + password to POGovApi	`PO_AUTH_LOGIN`	–	
3	Is the users list still current?		`PRO_POGOV_DOC_GET` **GET_STAMP users** (+ **GET_USERS** if it changed)	So several API servers see the same users
4	Is the Employee ID registered and active?		– (in memory)	**Not registered → step 4a**
4a	*(Not registered / inactive)* write the refusal		`PRO_POGOV_AUDIT` **APPEND** (`login-fail … not a registered employee`)	HRMS is **not** called. **End.**
5	Where is HRMS?		`PRO_GET_ALL_API` **SERVER_TASK** `HRP_GET_EMP_LOGIN_API` (kept 5 minutes)	HRMS URL of this environment
6	HRMS checks the password		– (HTTP call to HRMS `Api/Login/getLoginEmpDtl`)	Not in SQL Profiler

#	What happens	API task	Procedure (@Cmd)	Why
6a	*(Wrong password)* count the failure		`DOC_MAP_PUT` **USERS** (failed count / lock), `AUDIT` **APPEND** (`login-fail`, and `locked` after 5)	**End.**
7	Did anything change meanwhile?		`DOC_GET` **GET_STAMP users**, **GET_STAMP sessions**	
8	Save last sign-in, clear failures		`PRO_POGOV_DOC_MAP_PUT` **USERS**	
9	Create the session, end the other device's session		`PRO_POGOV_DOC_MAP_PUT` **SESSIONS**	One device at a time
10	Write "login"		`PRO_POGOV_AUDIT` **APPEND**	
11	Portal builds the sign-in cookie: "who is this?"	`PO_AUTH_ME`	`GET_STAMP users`, `GET_STAMP sessions` (+ `GET_USERS` / `GET_SESSIONS` because they just changed)	
12	... and which MCs this user may see		`DOC_GET` **GET_DOC manifest** → `BLOB_GET` **GET catheadmap** (only if the SCR file changed) → `DOC_GET` **GET_DOC scr_mapping**	SCR products file: buyer / category-head scope
13	Redirect to the role's start page	–	–	Section 3

2.3 On every screen, all the time

When	API task	Procedure (@Cmd)	Why
Every API call	(any)	`DOC_GET` **GET_STAMP sessions** + **GET_STAMP users** (+ `DOC_MAP_PUT SESSIONS` once every 5 minutes, to save "last seen")	Is the session still valid (not ended, not idle, not on another device)?
Every 60 s (session check of the open page)	`PO_AUTH_ME`	`GET_STAMP sessions`, `GET_STAMP users`, `GET_DOC manifest` (+ `GET_DOC scr_mapping`)	Forced sign-out when the admin changes the user (≤ 60 s)
Every 30 s (top bar: status, "new data" banner, approvals badge)	`PO_GET_STATUS`	`APP_LOCK GET` → `GET_DOC manifest`, `GET_DOC params`, `LOG_READ READ` (today) → `APP_LOCK RELEASE`; then `PO_AUTH_ME`-like user read (`GET_STAMP users`, scope)	Shows "New data on the server" and today's request number
... same call, **admin / planner / category head only**		`LOG_READ` **DAYS** + **READ_DAYS** (last 30 days)	Counts pending approvals for the ✅ badge. **The slowest repeating call (0.4–1 s).**

3. Role by role — what happens after sign-in

3.1 Buyer

#	Screen / action	API task	Procedure (@Cmd)	Why
1	Sign-in (section 2.2)	`PO_AUTH_LOGIN`, `PO_AUTH_ME`	as 2.2	
2	Lands on ✏️ **New Request**	`PO_GET_STATUS`	as 2.3 (**no** DAYS / READ_DAYS: buyers have no approvals badge)	
3	Today's requests list on the page	`PO_TOOL1_TODAY`	`LOG_READ` **READ** (requests, today)	"My requests today"
4	Types a Generic Material	`PO_TOOL1_OPTIONS`	`GET_DOC manifest` (data versions); `BLOB_GET` only if a data file changed since the last load	Colours / brand for the form; the data is kept in memory
5	**Validate**	`PO_TOOL1_VALIDATE`	`GET_DOC manifest`, `LOG_READ READ` (today, for headroom already used)	Decision in memory
6	**Submit**	`PO_TOOL1_SUBMIT`	`GET_DOC manifest`, `LOG_READ READ` (today) → scope check (`GET_DOC manifest`, `GET_DOC scr_mapping`) → `APP_LOCK GET` → `LOG_READ READ` (today, again under the lock) → `LOG_APPEND` (requests) → `APP_LOCK RELEASE` → `AUDIT APPEND` (`request`)	Saves the request; the second read under the lock makes sure nobody used the same headroom meanwhile

3.2 Category head

Same as the buyer, plus:

#	Screen / action	API task	Procedure (@Cmd)	Why
1	Status every 30 s	`PO_GET_STATUS`	as 2.3 **including** `LOG_READ DAYS` + `READ_DAYS`	Approvals badge (requests waiting for this person)
2	✅ **Approvals**	`PO_APPROVALS_GET`	`LOG_READ` **DAYS** + **READ_DAYS**	Watch List requests of the recent days
3	**Approve / Reject**	`PO_APPROVAL_DECIDE`	`APP_LOCK GET` → `LOG_READ READ` (that day) → `LOG_APPEND` (requests: approval event) → `APP_LOCK RELEASE` → `AUDIT APPEND` (`approval`)	

3.3 Planner

#	Screen / action	API task	Procedure (@Cmd)	Why
1	Sign-in	as 2.2		

#	Screen / action	API task	Procedure (@Cmd)	Why
2	Lands on 📂 **Data Setup**	`PO_GET_STATUS`	as 2.3 incl. DAYS / READ_DAYS	Data status per file, approvals badge
3	Uploads, parameters, history	see the **Data Setup process** document		
4	✅ **Approvals**	as 3.2		Planners can approve too

3.4 Admin (employee with the Admin role)

Everything a planner does (lands on Data Setup), plus:

#	Screen / action	API task	Procedure (@Cmd)	Why
1	👥 **Users**	`PO_USERS_LIST`	`GET_STAMP users` (+ `GET_USERS`), `GET_DOC manifest`, `GET_DOC scr_mapping`	List with MC counts
2	**Fetch from HRMS**	`PO_USER_HRMS_LOOKUP`	`PRO_GET_ALL_API` **SERVER_TASK** `HRP_GET_EMP_DETAIL_CODE` (kept 5 min) + HTTP to HRMS `getLoginEmpDtlByCode`; `GET_STAMP users`	Employee card
3	**Add user**	`PO_USER_ADD`	same HRMS lookup again → `DOC_MAP_PUT USERS` → `AUDIT APPEND` (`user-add`)	
4	**Edit: roles / Active**	`PO_USER_UPDATE`	`DOC_MAP_PUT USERS` → `GET_STAMP sessions` → `DOC_MAP_PUT SESSIONS` (the user's sessions marked ended) → `AUDIT APPEND` (`user-update`)	The user is signed out at their next check
5	**Audit log**	`PO_AUDIT_QUERY`	`PRO_POGOV_AUDIT` **READ**	

3.5 Built-in admin (email + local password)

Same as 2.2, except:

Step	Difference
5–6	**No HRMS**: the password is checked against the hash in `app_users` (no `SERVER_TASK`, no HTTP call)
First sign-in	Forced to **Change password**: `PO_AUTH_CHANGE_PASSWORD` → `DOC_MAP_PUT USERS` (new hash) → `DOC_MAP_PUT SESSIONS` (other sessions ended) → `AUDIT APPEND` (`password-change`) → `PO_AUTH_ME`

---

4. Leaving

Event	API task	Procedure (@Cmd)
⏻ **Sign out** (or idle 30 min)	`PO_AUTH_LOGOUT`	`GET_STAMP sessions` → `DOC_MAP_PUT SESSIONS` (session removed) → `AUDIT APPEND` (`logout`)
Signed out because of another device / admin change	`PO_AUTH_ME` (next check)	`GET_STAMP sessions` (changed) → `GET_SESSIONS` → answer 401 with the reason; then `PO_AUTH_LOGOUT` as above

---

5. Background calls (no user involved)

When	Who	Procedure	Why
POGovApi start	API	`SELECT CASE WHEN OBJECT_ID(…)` (schema check), `GET_STAMP/GET_USERS`, `GET_STAMP/GET_SESSIONS`	Load users and sessions into memory
POGovApi start + every 60 s	API (token refresh)	`PRO_GET_ALL_API` **ALL**	Tokens per API and environment
Portal start + every 5 min	Portal	`PRO_GET_ALL_API` **ALL** (through `PO_GET_ALL_API`)	The portal's API list
POGovApi start (once)	API (data warm-up)	`GET_DOC manifest`, `GET_DOC params`, `BLOB_GET` × every data file	Data files into memory before the first PO check
First SAP decision / every 5 min	API	`PRO_GET_ALL_API` **BASE_URL**	Portal address for the SAP verdict link

6. Example: the Profiler trace of 03-Nov-2026 08:42:45–08:43:23, read with this document

Time	What you see	Section
08:42:45	`GET_STAMP users`, `GET_USERS`, `GET_STAMP sessions`, `GET_SESSIONS`, `LOG_READ READ`	5: API start
08:42:49	`PRO_GET_ALL_API ALL` ×2, `BASE_URL`	5: portal + API lists
08:42:49–08:43:12	`GET_DOC manifest`, `BLOB_GET` × 12 (f3 7.4 s, artmaster 7.7 s)	5: data warm-up (slow network, not slow SQL)
08:42:57	`GET_STAMP users`, `SERVER_TASK HRP_GET_EMP_LOGIN_API`, `DOC_MAP_PUT USERS`, `DOC_MAP_PUT SESSIONS`, `AUDIT APPEND`	2.2 steps 3–10
08:42:58	`GET_USERS`, `GET_SESSIONS`, `GET_DOC manifest`, `BLOB_GET catheadmap`, `GET_DOC scr_mapping`	2.2 steps 11–12
08:42:58–59 and 08:43:19	`GET_DOC manifest`, `GET_DOC params`, `LOG_READ READ`, `DAYS`, `READ_DAYS` on two SPIDs together	2.3: status of the page + top bar (planner / admin)